

Centro Universitario de Ciencias Exactas e
Ingenierías (CUCEI)

SEMINARIO DE SOLUCION DE PROBLEMAS
DE TRADUCTORES DE LENGUAJES II

Tarea: Mini generador léxico

Mateo Sebastián serrano Pérez

25/01/2026



1. Introducción

En este proyecto se desarrolló un analizador léxico simple utilizando Python, el cual reconoce únicamente dos tipos de tokens: identificadores y números reales.

El programa recorre el código carácter por carácter usando un bucle infinito (`while True`) junto con una estructura tipo `switch` implementada mediante `if / elif / else`, tal como se indicó en las instrucciones del proyecto.

El objetivo principal fue comprender cómo funciona un analizador léxico desde cero, sin utilizar librerías externas para el reconocimiento de tokens, implementando manualmente las reglas de cada símbolo.

Este trabajo permitió reforzar conceptos vistos en clase como la lectura secuencial del código, el manejo de posiciones (línea y columna) y la detección de errores léxicos.

2. Objetivos

El objetivo general del proyecto es implementar un analizador léxico capaz de reconocer identificadores y números reales.

De manera específica, el programa debe reconocer identificadores siguiendo el patrón `letra(letra|digito)*` y números reales con el patrón `entero.entero+`. Además, se debe utilizar un bucle infinito junto con una estructura condicional tipo `switch` para procesar cada carácter del código.

El analizador también debe manejar errores léxicos y mostrar la línea y columna de cada token reconocido, ignorando correctamente los espacios en blanco y saltos de línea.

3. Especificación de Tokens

3.1 Identificadores

Los identificadores siguen el patrón `letra(letra|digito)*`. Esto significa que deben comenzar obligatoriamente con una letra y pueden continuar con letras o números. No está permitido que inicien con un dígito.

Algunos ejemplos válidos de identificadores son: `abc`, `variable1`, `x1y2`, `nombreVariable` y `ABC123`.

Ejemplos inválidos incluyen `1variable` y `_variable`, ya que no comienzan con una letra.

3.2 Números reales

Los números reales siguen el patrón `entero.entero+`. Primero debe existir una parte entera formada por uno o más dígitos, seguida de un punto decimal y al menos un dígito adicional como parte decimal.

Ejemplos válidos son `123.45`, `10.5`, `0.123`, `99.99` y `1000.0`.

Ejemplos inválidos son `123`, `.45`, `123.` y `12.34.56`.

4. Diseño e Implementación

El analizador léxico fue desarrollado en Python y está compuesto principalmente por tres clases. La clase `TokenType` define los tipos de tokens reconocidos por el sistema. La clase `Token` representa cada token, almacenando su tipo, valor, línea y columna. Finalmente, la clase `AnalizadorLexico` contiene toda la lógica principal del programa.

El método `analizar()` implementa el algoritmo principal. Primero se inicializan las variables necesarias y posteriormente se entra en un bucle infinito. Dentro del ciclo se ignoran los espacios en blanco y se obtiene el carácter actual. Dependiendo del tipo de carácter, se decide si se debe intentar reconocer un identificador o un número real. En caso contrario, el carácter se reporta como error.

Cada token reconocido se agrega a una lista y, al finalizar el análisis, se añade el token EOF.

La función `reconocer_identificador()` verifica que el token comience con una letra y posteriormente continúa leyendo mientras existan letras o dígitos. Por su parte, la función `reconocer_real()` primero lee la parte entera del número, después valida la existencia del punto decimal y finalmente verifica que exista al menos un dígito en la parte decimal.

5. Ejemplo de funcionamiento

Código de entrada

variable1 123.45 numero 67.89 abc123 10.5

Resultado del análisis

```
=====
ANALIZANDO CÓDIGO...
=====
ANALIZADOR LÉXICO SIMPLE
=====
TOKENS RECONOCIDOS:
=====
Tipo                Valor                Línea    Columna
-----
IDENTIFICADOR      variable1            1         1
REAL               123.45              1        11
IDENTIFICADOR      numero              1        18
REAL               67.89               1        25
IDENTIFICADOR      abc123              1        31
REAL               10.5                1        38
IDENTIFICADOR      temperatura          2         1
REAL               98.6                2        13
IDENTIFICADOR      velocidad            2        18
REAL               120.5               2        28
IDENTIFICADOR      pi                   3         1
REAL               3.14159             3         4
IDENTIFICADOR      resultado            3        12
REAL               42.0                3        22
EOF                $                   3        26
=====
Total de tokens: 15

Presiona Enter para salir..._
```

El analizador reconoce correctamente los identificadores variable1, numero y abc123, así como los números reales 123.45, 67.89 y 10.5. Además, muestra en pantalla una tabla con los tokens reconocidos junto con su línea y columna.

6. Casos de prueba

Se realizaron diferentes pruebas con identificadores simples, identificadores con dígitos, números reales y combinaciones de ambos. En todos los casos el analizador funcionó correctamente.

También se probaron entradas con números sin punto decimal y caracteres especiales, los cuales fueron detectados y reportados como errores, cumpliendo con el comportamiento esperado.

7. Manejo de errores

El analizador es capaz de detectar números sin punto decimal, caracteres no reconocidos y tokens mal formados. Cada error es reportado indicando la línea y columna donde ocurrió, lo cual facilita identificar el problema dentro del código de entrada.

Si se encuentra un número sin parte decimal, se muestra un mensaje indicando que el número no es válido. De igual forma, cualquier carácter que no sea letra, dígito o espacio en blanco es marcado como error.

8. Comparación con el código de referencia

El código de referencia proporcionado en clase presenta una implementación general de un analizador léxico utilizando estructuras similares como el uso de un bucle infinito y una lógica tipo switch para evaluar cada carácter.

Aunque el lenguaje utilizado en el material base es diferente, la lógica fue adaptada a Python, manteniendo el mismo enfoque de recorrido carácter por carácter y funciones auxiliares para el reconocimiento de tokens.

La principal diferencia es que el código de referencia reconoce más tipos de tokens, mientras que esta implementación se enfoca únicamente en identificadores y números reales, tal como lo solicita la actividad.

9. Limitaciones

Este analizador léxico únicamente reconoce identificadores y números reales. No acepta números enteros sin punto decimal ni operadores o símbolos especiales. Cualquier carácter fuera de estos casos es considerado un error.

Estas limitaciones fueron implementadas de manera intencional, ya que el alcance del proyecto se enfocó exclusivamente en los tokens solicitados.

10. Conclusión

Durante el desarrollo del proyecto se logró implementar correctamente un analizador léxico básico utilizando Python. Al inicio se presentaron algunas dificultades relacionadas con la validación de los números reales y el manejo de posiciones, pero conforme avanzó el desarrollo fue posible resolver estos detalles.

El programa cumple con los requisitos establecidos, reconoce correctamente identificadores y números reales, maneja errores léxicos y muestra la posición de cada token. Este proyecto permitió comprender mejor el funcionamiento interno de un analizador léxico y sirve como base para futuras implementaciones más completas.

11. Referencias

Repositorio de apoyo del curso:

<https://github.com/TraductoresLenguajes2/Traductores/tree/master/Modulo1/Compilador>

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.